# Travaux Pratiques
# Multi-Agent Systems avec SPADE

---

## Communication FIPA-ACL entre Agents

Version A

---


**Étudiant :** GOUNGOU Hamel


**Framework :** SPADE (Smart Python Agent Development Environment)
**Protocole :** FIPA-ACL (Foundation for Intelligent Physical Agents)
**Serveur XMPP :** PyJabber (serveur embarqué Python)
**Date :** 30 mars 2026

**Exercices couverts**

Exercice 1 : Échange d'informations (QUERY-IF / INFORM)
Exercice 2 : Directives d'action (REQUEST / AGREE / REFUSE / FAILURE)
Exercice 3 : Négociation Contract Net (CFP / PROPOSE / ACCEPT / REJECT)

# 0 Table des matières

**1 Exercice 1 — Échange d'Informations (QUERY-IF / INFORM) 2**
1.1 Scénario . . . 2
1.2 Flux de communication . . . 2
1.3 Implémentation — Agent Capteur . . . 2
1.4 Implémentation — Agent Moniteur . . . 2
1.5 Résultat obtenu . . . 3
1.6 Analyse . . . 3

**2 Exercice 2 — Directives d'Action (REQUEST / AGREE / REFUSE / FAILURE) 4**
2.1 Scénario . . . 4
2.2 Logique de décision du Robot . . . 4
2.3 Implémentation — Agent Robot . . . 4
2.4 Résultat obtenu . . . 5
2.5 Analyse . . . 6

**3 Exercice 3 — Négociation Contract Net (CFP / PROPOSE / ACCEPT / REJECT) 7**
3.1 Scénario . . . 7
3.2 Protocole Contract Net . . . 7
3.3 Implémentation — Worker Agent . . . 7
3.4 Implémentation — Monitor Agent (Ex3) . . . 8
3.5 Résultat obtenu . . . 9
3.6 Analyse . . . 9

**4 Conclusion 10**

# 1 Exercice 1 — Échange d'Informations (QUERY-IF / INFORM)

## 1.1 Scénario

Un **Agent Moniteur** interroge un **Agent Capteur** sur la température actuelle. Le Capteur filtre strictement les messages entrants grâce à un Template FIPA-ACL, et ne répond qu'aux requêtes de type `query-if`.

## 1.2 Flux de communication

$$\textbf{Monitor} \xrightarrow{\text{QUERY-IF}} \textbf{Sensor}$$
$$\textbf{Sensor} \xrightarrow{\text{INFORM}} \textbf{Monitor}$$

## 1.3 Implémentation — Agent Capteur

L'Agent Capteur utilise un `CyclicBehaviour` avec un Template strict pour ne réagir qu'aux messages `query-if` :

Listing 1 – sensor_agent.py

```python
from spade.agent import Agent
from spade.behaviour import CyclicBehaviour
from spade.template import Template

class SensorAgent(Agent):
    class BehaviorQuery(CyclicBehaviour):
        async def run(self):
            msg = await self.receive(timeout=10)
            if msg:
                print(f"[Sensor] Request received: {msg.body}")
                reply = msg.make_reply()
                reply.set_metadata("performative", "inform")
                temperature_actuelle = 25
                reply.body = f"The temperature is {temperature_actuelle}C"
                await self.send(reply)
                print("[Sensor] Reply (INFORM) sent.")

    async def setup(self):
        print("Sensor started.")
        template = Template()
        template.set_metadata("performative", "query-if")
        self.add_behaviour(self.BehaviorQuery(), template)
```

## 1.4 Implémentation — Agent Moniteur

L'Agent Moniteur utilise un `OneShotBehaviour` car l'action est unique (envoyer + recevoir) :

Listing 2 – monitor_agent.py

```python
from spade.agent import Agent
from spade.behaviour import OneShotBehaviour
from spade.message import Message

class MonitorAgent(Agent):
    class BehaviorAsk(OneShotBehaviour):
        async def run(self):
            msg = Message(to="sensor@localhost")
            msg.set_metadata("performative", "query-if")
            msg.body = "What is the current temperature?"
            await self.send(msg)
            print("[Monitor] QUERY-IF sent.")
            reply = await self.receive(timeout=10)
            if reply:
                print(f"[Monitor] Response received: {reply.body}")
            else:
                print("[Monitor] No response (timeout).")

    async def setup(self):
        print("Monitor started.")
        self.add_behaviour(self.BehaviorAsk())
```

## 1.5 Résultat obtenu

Listing 3 – Sortie console — Exercice 1

```
[Monitor] QUERY-IF sent.
[Sensor]  Request received: What is the current temperature?
[Sensor]  Reply (INFORM) sent.
[Monitor] Response received: The temperature is 25C
```

## 1.6 Analyse

Le Template sur l'Agent Capteur garantit un **filtrage strict** — tout message dont la performative n'est pas `query-if` est ignoré. Le choix du `OneShotBehaviour` pour le Moniteur reflète le caractère ponctuel de la requête, tandis que le `CyclicBehaviour` du Capteur lui permet de rester disponible en permanence.

# 2 Exercice 2 — Directives d'Action (REQUEST / AGREE / REFUSE / FAILURE)

## 2.1 Scénario

Un **Agent Moniteur** envoie des ordres de mouvement à un **Agent Robot autonome**. Contrairement à un objet passif, le Robot évalue son **état interne** (niveau de batterie) avant d'accepter ou de refuser l'ordre. Cinq mouvements successifs sont demandés pour couvrir tous les cas possibles.

## 2.2 Logique de décision du Robot

| Mouvement | Position | Batterie | Résultat |
|---|---|---|---|
| #1 | (5, 3) | 80% | AGREE + INFORM (succès) |
| #2 | (8, 1) | 65% | AGREE + FAILURE (obstacle) |
| #3 | (2, 6) | 50% | AGREE + INFORM (succès) |
| #4 | (4, 4) | 35% | AGREE + INFORM (succès) |
| #5 | (9, 2) | 20% | REFUSE (batterie insuffisante) |

Table 1 – Scénario des 5 mouvements — Version A

## 2.3 Implémentation — Agent Robot

Listing 4 – robot_agent.py

```python
class RobotAgent(Agent):
    def __init__(self, jid, password, battery_level=100):
        super().__init__(jid, password)
        self.battery_level = battery_level
        self.obstacle_on_move = 2  # le 2eme mouvement declenche un obstacle
        self.move_count = 0

    class BehaviorRequest(CyclicBehaviour):
        async def run(self):
            msg = await self.receive(timeout=15)
            if msg:
                self.agent.move_count += 1
                move_n = self.agent.move_count

                reply = msg.make_reply()
                if self.agent.battery_level > 20:
                    # AGREE : le robot accepte
                    reply.set_metadata("performative", "agree")
                    reply.body = f"Movement #{move_n} authorized."
                    await self.send(reply)

                    await asyncio.sleep(2)  # simulation du mouvement
```

```python

                    # Decharge la batterie apres chaque mouvement
                    self.agent.battery_level -= 15

                    # INFORM ou FAILURE selon l'obstacle predefini
                    inform = msg.make_reply()
                    obstacle = (move_n == self.agent.obstacle_on_move)
                    if not obstacle:
                        inform.set_metadata("performative", "inform")
                        inform.body = f"Movement #{move_n} completed. Battery: {self.agent.battery_level}%"
                    else:
                        inform.set_metadata("performative", "failure")
                        inform.body = f"Movement #{move_n} failed: obstacle detected."
                    await self.send(inform)
                else:
                    # REFUSE : batterie trop faible
                    reply.set_metadata("performative", "refuse")
                    reply.body = f"Refused: battery too low ({self.agent.battery_level}%)."
                    await self.send(reply)
```

## 2.4 Résultat obtenu

Listing 5 – Sortie console — Exercice 2 (mouvements 1 a 5)

```
[Monitor] Sending REQUEST #1: Move to position (5, 3)
[Robot]   REQUEST #1 received     Battery: 80%
[Monitor] Response #1 (AGREE): Movement #1 authorized. Execution in progress.
[Robot]   Battery after movement: 65%
[Robot]   INFORM (success) sent for movement #1.
[Monitor] Result #1 (INFORM): Movement #1 completed successfully. Battery: 65%

[Monitor] Sending REQUEST #2: Move to position (8, 1)
[Robot]   REQUEST #2 received     Battery: 65%
[Monitor] Response #2 (AGREE): Movement #2 authorized. Execution in progress.
[Robot]   Battery after movement: 50%
[Robot]   FAILURE (obstacle) sent for movement #2.
[Monitor] Result #2 (FAILURE): Movement #2 failed: obstacle detected. Battery: 50%

[Monitor] Sending REQUEST #3: Move to position (2, 6)
[Robot]   REQUEST #3 received     Battery: 50%
[Monitor] Response #3 (AGREE): Movement #3 authorized. Execution in progress.
[Robot]   Battery after movement: 35%
[Robot]   INFORM (success) sent for movement #3.
[Monitor] Result #3 (INFORM): Movement #3 completed successfully. Battery: 35%
```

```
[Monitor] Sending REQUEST #4: Move to position (4, 4)
[Robot]   REQUEST #4 received     Battery: 35%
[Monitor] Response #4 (AGREE): Movement #4 authorized. Execution in
   progress.
[Robot]   Battery after movement: 20%
[Robot]   INFORM (success) sent for movement #4.
[Monitor] Result #4 (INFORM): Movement #4 completed successfully.
   Battery: 20%

[Monitor] Sending REQUEST #5: Move to position (9, 2)
[Robot]   REQUEST #5 received     Battery: 20%
[Robot]   REFUSE sent for movement #5 (battery: 20%).
[Monitor] Response #5 (REFUSE): Movement #5 refused: battery too low
   (20%).

[Monitor] All movement requests completed.
```

## 2.5 Analyse

Cet exercice démontre l'**autonomie** de l'agent Robot : il ne se contente pas d'exécuter les ordres mécaniquement, mais évalue son état interne avant de répondre. Les trois cas FIPA-ACL sont couverts :

- **AGREE + INFORM** : ordre accepté et exécuté avec succès (mouvements 1, 3, 4)
- **AGREE + FAILURE** : ordre accepté mais échec à l'exécution (mouvement 2 — obstacle)
- **REFUSE** : ordre refusé dès la réception car la batterie est insuffisante (mouvement 5)

# 3 Exercice 3 — Négociation Contract Net (CFP / PROPOSE / ACCEPT / REJECT)

## 3.1 Scénario

L'**Agent Moniteur** doit déléguer une tâche lourde (*CNN Training*) à l'un de ses Workers. Il lance un **appel d'offres** (CFP) auprès de deux Workers, compare leurs propositions, puis sélectionne le moins coûteux.

## 3.2 Protocole Contract Net

$$\textbf{Monitor} \xrightarrow{\text{CFP}} \textbf{Worker1} \qquad \textbf{Monitor} \xrightarrow{\text{CFP}} \textbf{Worker2}$$

$$\textbf{Worker1} \xrightarrow{\text{PROPOSE (10 units)}} \textbf{Monitor}$$

$$\textbf{Worker2} \xrightarrow{\text{PROPOSE (5 units)}} \textbf{Monitor}$$

$$\textbf{Monitor} \xrightarrow{\text{ACCEPT-PROPOSAL}} \textbf{Worker2} \qquad \text{(moins cher)}$$

$$\textbf{Monitor} \xrightarrow{\text{REJECT-PROPOSAL}} \textbf{Worker1}$$

## 3.3 Implémentation — Worker Agent

Chaque Worker possède deux comportements distincts : un pour recevoir le CFP, un autre pour recevoir la décision finale. Les deux sont séparés par des Templates spécifiques.

Listing 6 – worker_agent.py

```python
class WorkerAgent(Agent):
    def __init__(self, jid, password, cost):
        super().__init__(jid, password)
        self.cost = cost

    class BehaviorCFP(CyclicBehaviour):
        async def run(self):
            msg = await self.receive(timeout=10)
            if msg:
                reply = msg.make_reply()
                reply.set_metadata("performative", "propose")
                reply.body = f"Cost: {self.agent.cost} time units"
                await self.send(reply)

    class BehaviorDecision(CyclicBehaviour):
        async def run(self):
            msg = await self.receive(timeout=15)
            if msg:
                perf = msg.get_metadata("performative")
                print(f"[Worker {self.agent.jid.local}] Decision: {perf.upper()}")

    async def setup(self):
        template_cfp = Template()
```

```
        template_cfp.set_metadata("performative", "cfp")
        self.add_behaviour(self.BehaviorCFP(), template_cfp)

        template_accept = Template()
        template_accept.set_metadata("performative", "accept-proposal")
        template_reject = Template()
        template_reject.set_metadata("performative", "reject-proposal")
        # Operateur | : ecoute accept OU reject
        self.add_behaviour(self.BehaviorDecision(), template_accept |
    template_reject)
```

## 3.4 Implémentation — Monitor Agent (Ex3)

Listing 7 – monitor_ex3.py — selection par cout minimal

```
class BehaviorCFP(OneShotBehaviour):
    async def run(self):
        workers = ["worker1@localhost", "worker2@localhost"]

        # 1. Envoi du CFP aux deux workers
        for w in workers:
            cfp = Message(to=w)
            cfp.set_metadata("performative", "cfp")
            cfp.body = "CNN Training"
            await self.send(cfp)

        # 2. Collecte des propositions (template filtre sur "propose")
        proposals = []
        for _ in workers:
            proposal = await self.receive(timeout=15)
            if proposal:
                cost = int(proposal.body.split(":")[1].strip().split()
    [0])
                proposals.append((proposal, cost))

        # 3. Tri par cout croissant     le moins cher gagne
        proposals.sort(key=lambda x: x[1])
        best = proposals[0][0]
        others = [p[0] for p in proposals[1:]]

        # 4. ACCEPT au gagnant
        accept = best.make_reply()
        accept.set_metadata("performative", "accept-proposal")
        accept.body = "Your proposal is accepted. Please start CNN
    Training."
        await self.send(accept)

        # 5. REJECT aux autres
        for loser in others:
            reject = loser.make_reply()
            reject.set_metadata("performative", "reject-proposal")
            reject.body = "Your proposal was not selected."
            await self.send(reject)
```

## 3.5 Résultat obtenu

Listing 8 – Sortie console — Exercice 3

```
[Monitor] CFP sent to worker1@localhost
[Monitor] CFP sent to worker2@localhost
[Worker worker1] CFP received: CNN Training
[Worker worker1] PROPOSE sent -- Cost: 10 time units
[Worker worker2] CFP received: CNN Training
[Worker worker2] PROPOSE sent -- Cost: 5 time units
[Monitor] PROPOSE received from worker1@localhost: Cost: 10 time units
[Monitor] PROPOSE received from worker2@localhost: Cost: 5 time units
[Monitor] ACCEPT-PROPOSAL sent to worker2@localhost
[Monitor] REJECT-PROPOSAL sent to worker1@localhost
[Worker worker2] Decision received: ACCEPT-PROPOSAL
[Worker worker1] Decision received: REJECT-PROPOSAL
```

## 3.6 Analyse

Le protocole **Contract Net** est une implémentation directe du mécanisme d'enchères inversées de FIPA. Points clés :

- Le Template `propose` sur le Monitor garantit qu'il n'écoute que les réponses pertinentes
- L'opérateur `|` (OU logique) entre Templates permet au Worker d'écouter `accept-proposal` **ou** `reject-proposal` avec un seul behaviour
- Le critère de sélection est le **coût minimal** : Worker2 (5 unités) est préféré à Worker1 (10 unités)

# 4 Conclusion

Ce TP a permis de mettre en pratique les trois grandes familles de performatives FIPA-ACL dans un système multi-agents réel basé sur SPADE :

1. **Échange d'informations** : le protocole QUERY-IF / INFORM permet une communication simple et directe entre un agent interrogateur et un agent fournisseur d'information.
2. **Directives d'action** : le trio REQUEST / AGREE / REFUSE illustre parfaitement le concept d'**autonomie** des agents — le Robot ne se contente pas d'obéir, il évalue son état interne et peut refuser un ordre. Le FAILURE complète ce tableau en signalant un échec à l'exécution.
3. **Négociation Contract Net** : le protocole CFP / PROPOSE / ACCEPT-PROPOSAL / REJECT-PROPOSAL implémente un mécanisme d'allocation de ressources décentralisé où le meilleur prestataire est sélectionné objectivement.

L'utilisation des **Templates FIPA-ACL** de SPADE est un élément clé : ils garantissent que chaque agent ne traite que les messages qui lui sont destinés, évitant ainsi les interférences entre les différents protocoles de communication.